🚀 Master Full-Stack Development & Deployment Guide

INSTRUCTOR SLIDES

Complete 11-Step Pipeline from Setting up Antigravity IDE to Live Production Deployment

1. **Antigravity IDE Setup**
   Download, launch, open empty workspace, verify terminal & AI prompt
2. **Write the Mega Prompt**
   Architect complete app (UI + Server + DB schema + API) in one prompt
3. **Set Up Supabase (DB & Auth)**
   Create DB, get Project URL + keys, execute database.sql
4. **Get Gemini / Groq API Key**
   Google AI Studio API key generation for AI features
5. **Project Structure & .env Config**
   client/ & server/ folders, configure environment files & strict .gitignore
6. **Local Testing & Verification**
   Run client (5173) and server (5000) concurrently
7. **Git Initialization & GitHub Push**
   git init, commit, branch main, link remote repo & push safely
8. **Deploy Backend on Render**
   Host Node.js server with Root Dir server, set env vars
9. **Deploy Frontend on Vercel**
   Connect repo, Root Dir client, set VITE_API_BASE_URL
10. **Automatic Linking (CORS & Base URL)**
    Frontend talks to Render backend over HTTPS automatically
11. **End-to-End Live Verification**
    Auth test, Database CRUD test, AI advisory response check

1 Step 1 — Setting Up Google Antigravity IDE

Your AI-native pairing environment for building full-stack applications

1.1 Install & Launch Antigravity

Download and install Antigravity IDE on Windows/macOS/Linux. Launch the IDE.

1.2 Open / Create a Workspace Folder

Go to **File** → **Open Folder** and choose your project folder (e.g. `C:projectsmy-app`). The agent will anchor all file operations here.

1.3 Verify Terminal & PowerShell Access

Press `Ctrl + ` ` to ensure integrated terminal works (Node.js & git available).

1.4 Select AI Model & Start Agent Session

Open the Agent Chat panel. Select your preferred reasoning model (e.g., Gemini 3.8 Flash or Claude Opus). You are now ready to issue your mega prompt!

```
# Prerequisites Check in Antigravity Terminal:
node -v # Recommended: Node v18+ or v20+
npm -v # Recommended: npm v9+ or v10+
git --version # Git installed & configured
```

💡 **Antigravity Agent Capabilities:**
Antigravity doesn't just write snippets — it creates files directly on disk, runs commands in terminal, modifies configs, and debugs errors automatically.

⚠️ **Sandbox Permissions:**
Antigravity runs commands in a secure sandbox by default. Commands needing external network access (like `git push`) will prompt for your authorization.

2 Step 2 — Crafting the Mega Prompt

Architect the entire project in a single comprehensive instruction

5 Key Elements of a Winning Mega Prompt:

1. Role & Application Goal: "Act as an expert full-stack developer. Build an AI-powered agricultural advisory platform called CropAI."

2. Tech Stack Definition: "Frontend: React + Vite + Tailwind CSS. Backend: Node.js + Express. Database: Supabase. AI: Gemini API."

3. Explicit Folder Architecture: "Create client/ and server/ folders. Include root .gitignore, .env.example, and database.sql."

4. API Contract & Middleware: "Implement CORS for *.vercel.app, rate limiting, helmet security headers, and routes for /api/farms and /api/advisory."

5. Clean UI & Auth Flow: "Modern responsive dashboard with Supabase email authentication and farm management."

Template to Copy & Paste:

```
"Build a production-grade full-stack app called <AppName>.
Structure:
- /client: React (Vite) + Tailwind CSS + Lucide Icons
- /server: Express.js REST API with CORS configured for Vercel
- /database.sql: PostgreSQL schema with RLS for Supabase
- .gitignore: Exclude all .env and node_modules
Features:
1. User login/signup with Supabase Auth
2. Full CRUD operations backed by database
3. AI integration using Gemini API with structured JSON output
4. Responsive dashboard UI with dark/light aesthetics"
```

💡 **Instructor Tip:** Always tell the AI to create `.env.example` with dummy placeholders so students know exactly which keys are needed without leaking credentials.

3 Step 3 — Supabase Database & Auth Setup

Provisioning PostgreSQL database and Authentication in under 2 minutes

3.1 Create Project at supabase.com

Click **New Project** → Set Name, Secure Database Password, and select closest region.

3.2 Execute SQL Schema

Navigate to **SQL Editor** → Paste contents of `database.sql` generated by Antigravity → Click **Run**.

3.3 Retrieve API Credentials (Project Settings → API)

Credential	Where to Paste	Purpose	Security
`Project URL`	`client/.env & server/.env`	`Supabase endpoint URL`	`Public`
`anon Public Key`	`client/.env (VITE_SUPABASE_ANON_KEY)`	`Client-side auth & queries`	`Safe for browser`
`service_role Key`	`server/.env (SUPABASE_SERVICE_ROLE_KEY)`	`Backend admin DB access`	**`CRITICAL SECRET`**

> ⚠️ **Security Golden Rule:** Never put the `service_role` key inside the `client/` folder. The frontend bundles into public JS — anyone could view page source and obtain full admin access to your database!

4 Step 4 — Generating Google Gemini API Key

Powering your backend with state-of-the-art AI inference

4.1 Visit Google AI Studio

Go to `aistudio.google.com` and sign in with your Google Account.

4.2 Create API Key

Click **Get API Key** in the sidebar → Click **Create API Key** in new or existing GCP project.

4.3 Copy & Store Key

Copy your key string (e.g. `AQ...` or `AIzaSy...`). Add to `server/.env`.

```
# Add inside server/.env:
GEMINI_API_KEY=AIzaSy...your_gemini_api_key_here
```

> 💡 **Free Tier Perks:**
> Google Gemini gives a generous free quota (up to 15 Requests Per Minute on Gemini 1.5/2.0 Flash) — perfect for student projects and hackathons!

> 💡 **Backup AI Engine (Groq):**
> If Gemini rate-limits during live demo, you can get a free fast inference key from `console.groq.com/keys` and use Llama/Qwen models.

5 Step 5 — Project Structure & Environment Isolation

Strict file separation prevents credential leaks and build failures

```
root-project-directory/
├── client/ (Frontend - Vite)
│   ├── src/
│   ├── .env (VITE_ prefixed only)
│   ├── package.json
│   └── vite.config.js
├── server/ (Backend - Express)
│   ├── routes/ & controllers/
│   ├── .env (Secret API keys)
│   ├── index.js
│   └── package.json
├── .gitignore (CRITICAL - MUST IGNORE .env)
├── .env.example (Safe placeholders for repo)
└── database.sql
```

client/.env:

```
VITE_SUPABASE_URL=https://xyz.supabase.co
VITE_SUPABASE_ANON_KEY=eyJ...
VITE_API_BASE_URL=http://localhost:5000/api
```

server/.env:

```
PORT=5000
SUPABASE_URL=https://xyz.supabase.co
SUPABASE_SERVICE_ROLE_KEY=eyJ...
GEMINI_API_KEY=your_key
FRONTEND_URL=http://localhost:5173
```

6 Step 6 — Running & Testing Locally

Always verify both client and server before pushing to cloud providers

Terminal 1: Start Express Server

```
cd server
npm install
npm run dev
# Output: Server running on port 5000
```

Terminal 2: Start Vite Client

```
cd client
npm install
npm run dev
# Output: Local: http://localhost:5173
```

4-Point Pre-Deployment Local Check:

1. **Health Check:** Visit `http://localhost:5000/health` → Expect `{"status":"ok"}`
2. **User Sign Up:** Register test account at `http://localhost:5173`
3. **DB Persistence:** Add new item (e.g. Farm) and refresh to verify data persists
4. **AI Route:** Click "Generate Advisory" and confirm Gemini output renders

7 Step 7 — Git Commands & Pushing to GitHub

Setting up version control to trigger auto-deployments on Render and Vercel

Initial Setup (Run Once in Root Folder):

```
# 1. Initialize git in project root
git init

# 2. Stage files (verifies .gitignore)
git add .

# 3. Create initial commit
git commit -m "Initial commit: Full stack application"

# 4. Rename default branch to main
git branch -M main

# 5. Link GitHub repo (create empty repo on GitHub first)
git remote add origin https://github.com/USER/REPO.git

# 6. Push code to GitHub
git push -u origin main
```

Subsequent Updates (Whenever Code Changes):

```
git add .
git commit -m "Describe what you added/fixed"
git push origin main
```

> ⚠️ **GitHub Push Protection Warning:**
> GitHub will automatically reject your push with `GH013: Push cannot contain secrets` if an API key is accidentally staged in `.env.example`! Always use dummy placeholders.

8 Step 8 — Deploying Backend to Render

Hosting the Node.js / Express API server on Render's free cloud tier

Web Service Configuration Settings:

Render Field	Value	Notes
Name	cropai-backend	Generates your onrender.com URL
Root Directory	server	MUST be set to server!
Runtime	Node	Node.js environment
Build Command	npm install	Installs backend modules
Start Command	node index.js	Runs server entry point
Instance Type	Free	512 MB RAM, free tier

Render Environment Variables (Dashboard → Environment):

Variable Key	Value to Enter
PORT	5000
SUPABASE_URL	https://your-project.supabase.co
SUPABASE_SERVICE_ROLE_KEY	eyJ... (from Supabase API settings)
GEMINI_API_KEY	Your Gemini Studio API Key
FRONTEND_URL	https://your-frontend.vercel.app

> 💡 **Verify Live Backend:**
> Once deployed, open: `https://<app>.onrender.com/health`
> If it returns `{"status":"ok"}`, your backend is 100% operational!

9 Step 9 & 10 — Deploying Frontend to Vercel & Linking

Hosting Vite React UI and connecting it directly to your Render API

Vercel Project Settings:

Vercel Field	Value
Framework Preset	Vite
Root Directory	client (Click Edit)
Build Command	npm run build
Output Directory	dist

> 💡 **The "Deploy Button" on Vercel:**
> Vercel doesn't need a manual deploy button — whenever you push code with `git push origin main`, Vercel detects it and triggers a fresh build automatically!

Vercel Environment Variables (Settings → Environment Variables):

Key	Value
VITE_SUPABASE_URL	https://your-project.supabase.co
VITE_SUPABASE_ANON_KEY	eyJ... (Supabase anon key)
VITE_API_BASE_URL	https://your-backend.onrender.com/api

> 🔗 **How Render & Vercel Connect:**
> You DO NOT need any special dashboard plugin between Render and Vercel. Setting `VITE_API_BASE_URL` tells the browser to route requests directly to Render!

11 Step 11 — The Ultimate Instructor Demonstration Checklist

Follow this sequence in class or presentation to show a 100% working live system

1. Antigravity IDE: Workspace opened, agent session initialized, terminal verified	**READY ✓**
2. Security Check: .gitignore excludes `.env`; no live secrets in `.env.example`	**READY ✓**
3. Supabase: Tables created in SQL Editor, Auth enabled, anon & service_role keys stored	**READY ✓**
4. Render Backend: Root Dir = `server`, env vars populated, `/health` endpoint returns OK	**READY ✓**
5. Vercel Frontend: Root Dir = `client`, `VITE_API_BASE_URL` set to Render API	**READY ✓**
6. Live End-to-End Demo: Sign up user → Create entity in DB → Query AI API → Inspect clean Console (F12)	**READY ✓**

🎉 **Congratulations! Your Full-Stack Cloud Architecture is Complete.**

Both frontend and backend are hooked into GitHub CI/CD: every push to `main` auto-deploys to production.